PROBLEM SOLVING HOME | ANOTHER PROBLEM LIKE THIS | HIDE QUESTIONS | HIDE SOLUTION

INTERACTIVE END-OF-CHAPTER EXERCISES

RELIABLE DATA TRANSFER: RDT2.2 (SENDER AND RECEIVER ACTIONS)

Consider the rdt2.2 protocol from the text (pages 209-212). The FSMs for the sender and receiver are shown below:

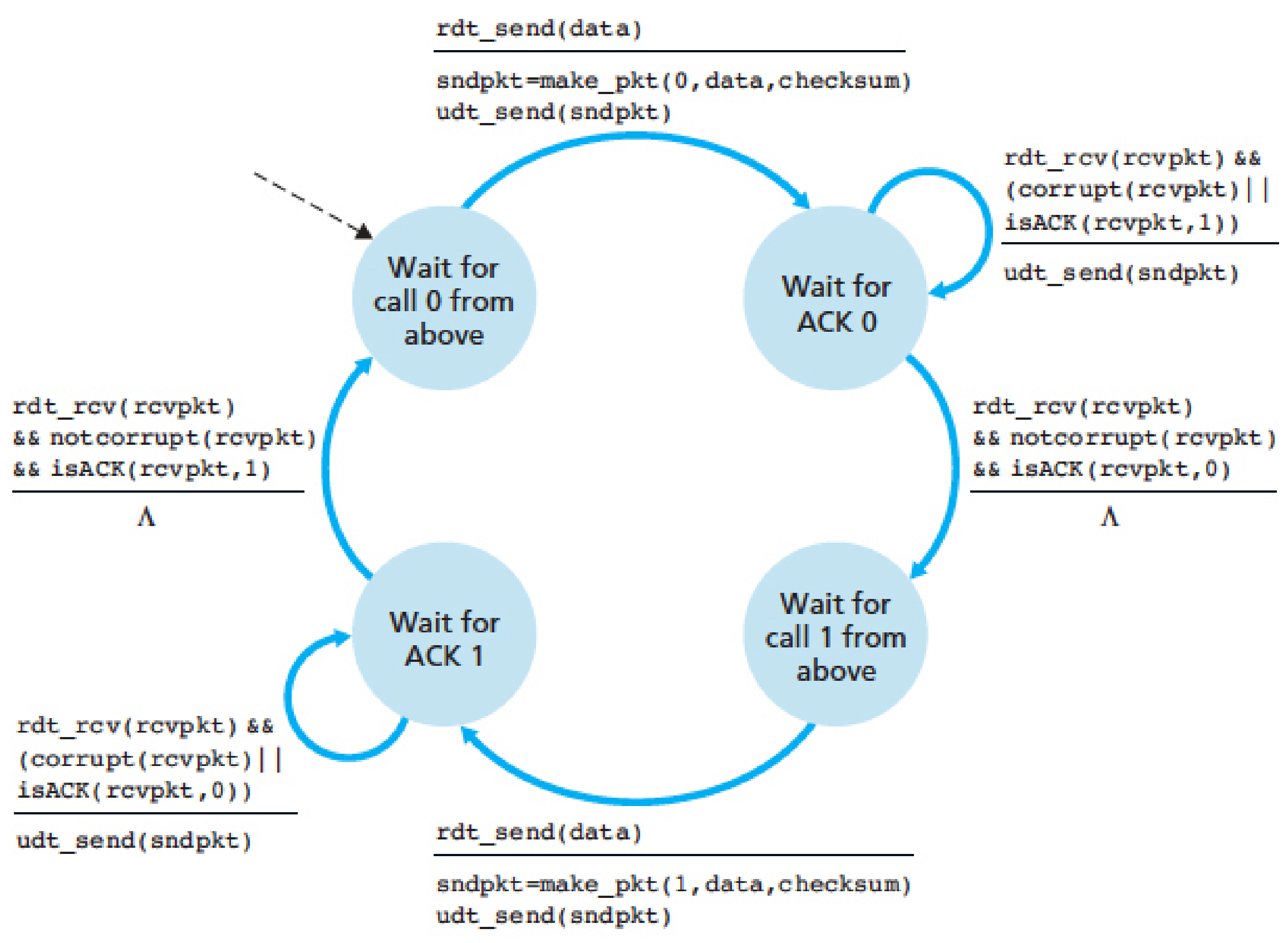

rdt_send(data)
sndpkt=make_pkt(0,data,checksum)
udt_send(sndpkt)
Wait for call 0 from above
Wait for ACK 0
rdt_rcv(rcvpkt) && (corrupt(rcvpkt)|| isACK(rcvpkt,1))
udt_send(sndpkt)
rdt_rcv(rcvpkt) && notcorrupt(rcvpkt) && isACK(rcvpkt,1)
Λ
rdt_rcv(rcvpkt) && notcorrupt(rcvpkt) && isACK(rcvpkt,0)
Λ
Wait for ACK 1
Wait for call 1 from above
rdt_rcv(rcvpkt) && (corrupt(rcvpkt)|| isACK(rcvpkt,0))
udt_send(sndpkt)
rdt_send(data)
sndpkt=make_pkt(1,data,checksum)
udt_send(sndpkt)

Figure 3.13 ♦ `rdt2.2` sender

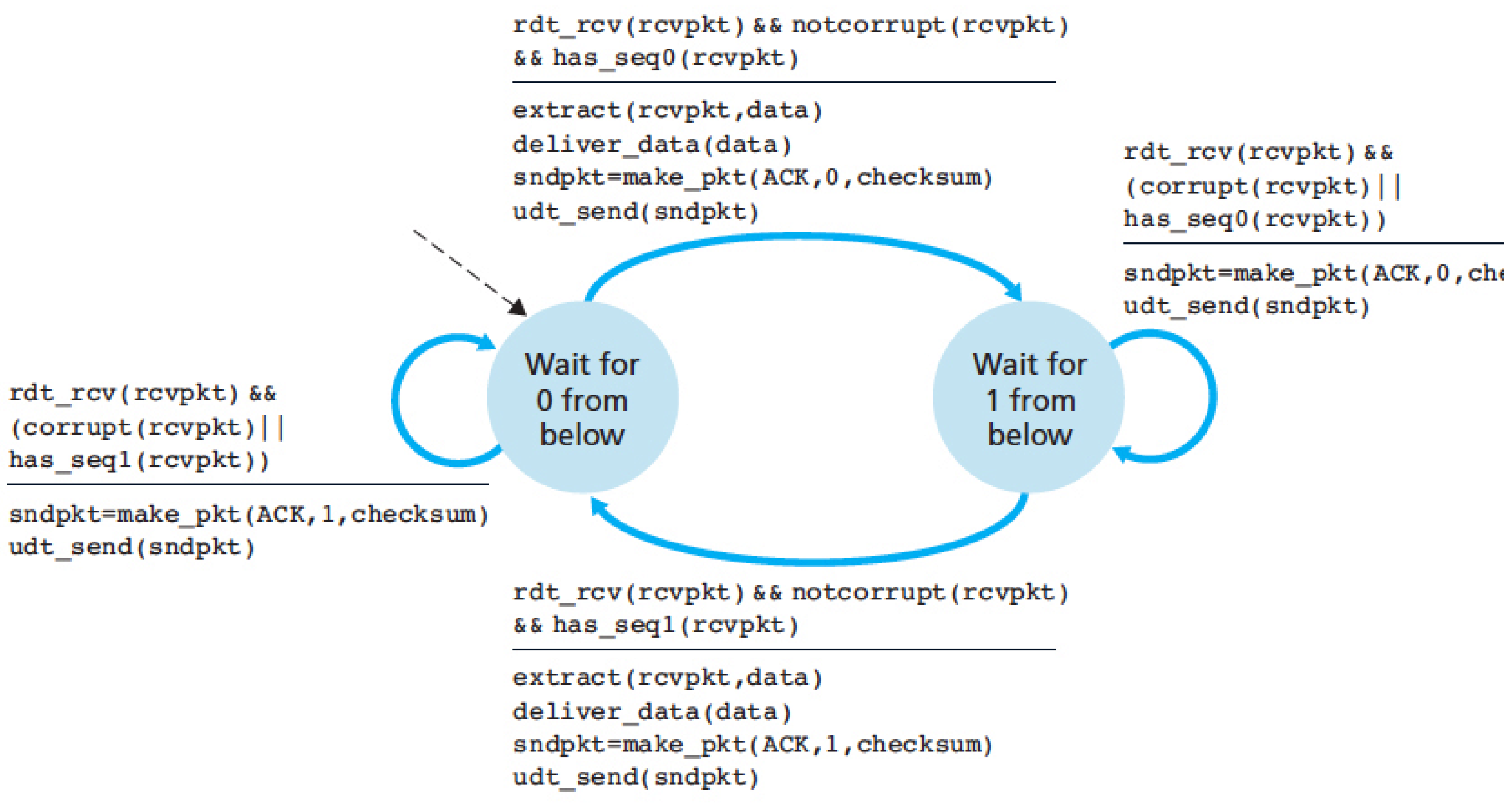
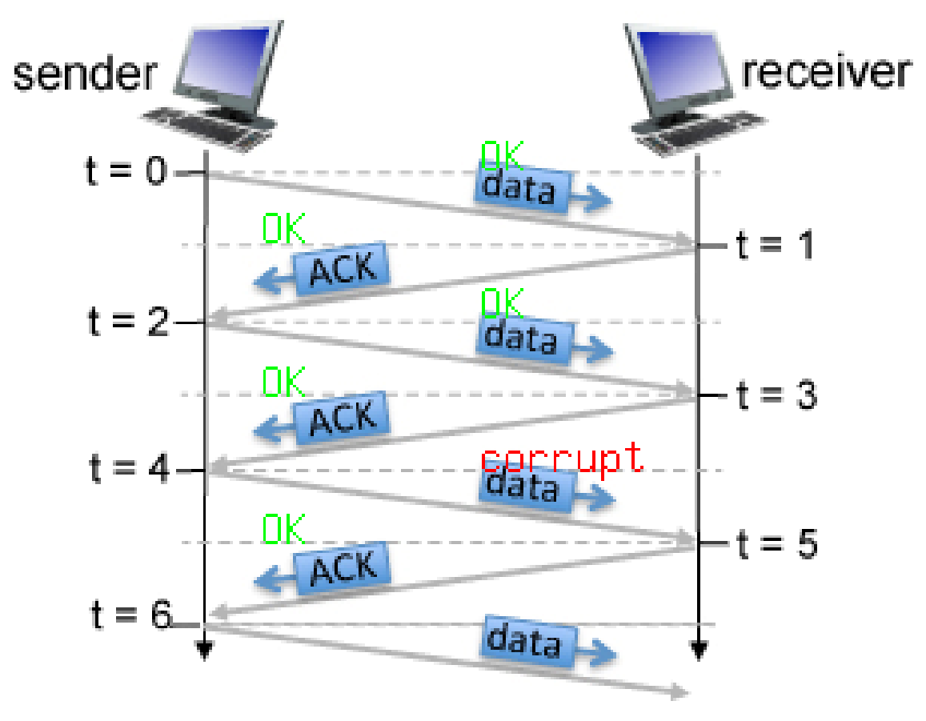

rdt_rcv(rcvpkt) && notcorrupt(rcvpkt) && has_seq0(rcvpkt)
extract(rcvpkt,data)
deliver_data(data)
sndpkt=make_pkt(ACK,0,checksum)
udt_send(sndpkt)
rdt_rcv(rcvpkt) && (corrupt(rcvpkt)|| has_seq0(rcvpkt))
sndpkt=make_pkt(ACK,0,ch
udt_send(sndpkt)
rdt_rcv(rcvpkt) && (corrupt(rcvpkt)|| has_seq1(rcvpkt))
sndpkt=make_pkt(ACK,1,checksum)
udt_send(sndpkt)
Wait for 0 from below
Wait for 1 from below
rdt_rcv(rcvpkt) && notcorrupt(rcvpkt) && has_seq1(rcvpkt)
extract(rcvpkt,data)
deliver_data(data)
sndpkt=make_pkt(ACK,1,checksum)
udt_send(sndpkt)

Figure 3.14 ♦ `rdt2.2` receiver

Suppose that the channel connecting the sender and receiver can corrupt but not lose or reorder packets. Now consider the figure below, which shows four data packets and three corresponding ACKs being exchanged between an rdt 2.2 sender and receiver. The actual corruption or successful transmission/reception of a packet is indicated by the corrupt and OK labels, respectively, shown above the packets in the figure below.



QUESTION LIST

1. At time t=0, what is the sender state?
2. At time t=0, what is the receiver state?
3. At time t=0, what is the sequence/ack # of the packet?
4. At time t=1, what is the sender state?
5. At time t=1, what is the receiver state?
6. At time t=1, what is the sequence/ack # of the packet?
7. At time t=2, what is the sender state?
8. At time t=2, what is the receiver state?
9. At time t=2, what is the sequence/ack # of the packet?
10. At time t=3, what is the sender state?
11. At time t=3, what is the receiver state?
12. At time t=3, what is the sequence/ack # of the packet?
13. How many times is the payload of the received packet passed up to the higher layer?

SOLUTION

1. At time t=0, the sender state is: Wait for ACK 0
2. At time t=0, the receiver state is: Wait for 0 from below
3. At time t=0, the sequence # is: 0
4. At time t=1, the sender state is: Wait for ACK 0
5. At time t=1, the receiver state is: Wait for 1 from below
6. At time t=1, the ACK # is: 0
7. At time t=2, the sender state is: Wait for ACK 1
8. At time t=2, the receiver state is: Wait for 1 from below
9. At time t=2, the sequence # is: 1
10. At time t=3, the sender state is: Wait for ACK 1
11. At time t=3, the receiver state is: Wait for 0 from below
12. At time t=3, the ACK # is: 1
13. 2 packets were passed up to the higher layer by the receiver.

QUESTION 1 OF 13

At time t=0, what is the sender state?

Answer

CHECK HINT SKIP

We greatly appreciate the work of John Broderick (UMass '21) in helping to develop these interactive problems.

Copyright © 2010-2025 J.F. Kurose, K.W. Ross

Comments welcome and appreciated: kurose@cs.umass.edu